Basic Machine Project — Team Design Note

1. Project Overview

The Basic Machine project simulates a simple von Neumann–style computer architecture using Java. It consists of: a two-pass assembler (Part 0), a basic single-port memory + CPU simulation (Part 1), a JavaFX user interface, and an automated test suite. The goal is to translate assembly source code into machine code, load it into memory, and execute it instruction by instruction within a virtual CPU.

2. System Architecture

The system consists of multiple coordinated modules, each with defined responsibilities:
Assembler: Translates assembly into binary machine code (two-pass, with symbol table).
Memory Module: Simulates 2048 × 16-bit memory using MAR/MBR interface.
CPU Module: Decodes and executes instructions (LDR, STR, LDA, LDX, JZ, HLT).
Simulator (JavaFX): Provides UI for program execution and visualization.
Program Loader: Loads .ld or .lst files into memory, determines program start.
Testing Framework: Validates assembler, CPU, and simulator via automated tests.

3. Team Responsibilities

Person A – Assembler and file generation; Person B – CPU and registers; Person C – GUI and UI event control; Person D – Testing and automation.

4. Design Decisions

• Instruction format: 16-bit [opcode(6), R(2), IX(2), I(1), address(5)]
• Memory: single-port, 2048 words
• Execution cycle: fetch–decode–execute
• EA calculation: supports indexed and indirect addressing
• Unified opcode mapping across assembler and CPU
• GUI displays all registers, memory cache, and allows step/run/halt/load operations.

5. Integration Flow

Assembler produces load.ld → ProgramLoader loads → MachineCore executes → GUI displays results. Each component communicates through shared Register and Memory modules.

6. Automated Testing

The test suite includes MemoryTest, CPUInstructionTest, IntegrationTest, and SimulatorRunTest. run_all.sh compiles, tests, and launches the simulator automatically.

7. Deployment

To build and run:
1. Build JAR: bash build_jar.sh
2. Run all tests + simulator: bash run_all.sh

Requirements: Java 21+, JavaFX SDK 25, and source.src in the same directory.

8. Conclusion

This project demonstrates the full life cycle of a simple computer system — assembling, loading, executing, and visualizing instructions. It emphasizes modular design, testability, and UI interactivity, providing a foundation for further extensions like floating-point or vector operations.